

TS 补全实测 · 统计与踩坑记录

柯里 · 公众号「前端 × AI 实战」 | 对象：137 个 JS 文件、约 3 万行、TS 覆盖率 0 的 Vue2 老后台

一、实测数据

指标	数值
改造文件数	137 个 → 116 个转 .ts/.vue（21 个第三方 demo 排除）
首轮编译通过率	约 6 成文件一次过
any 残留（改造后）	412 处，集中在接口字段与第三方库
过程累计编译报错	约 900+ 条，AI 自修约 7 成，3 成需人工
总耗时	约 40 小时（含 review），约为手写的 1/3~1/4

二、AI 反复翻车的四类类型

- 泛型：爱写 (obj: any, key: string) => any，不会主动推导。给它"标准答案"后能照着写。
- 老项目 this / 全局变量：this.\$http、window 全局配置无从推断，会编类型或给 any。
- 第三方库无类型：直接 declare module 'xxx' 整模块 any。应给"用到哪个导出声明哪个"的最小声明。
- 复杂接口：会脑补后端根本没返回的字段。只声明代码用到的字段，禁止猜测。

三、给 AI 的标准答案（开工前先喂）

```
# 取对象字段 — 用泛型，不要用 any
function pick<T, K extends keyof T>(obj: T, key: K): T[K] {
  return obj[key]
}
# 接口只声明代码用到的字段，禁止补后端没返回的字段
```

四、global.d.ts 模板（全局声明先行）

```
interface Window {
  APP_CONFIG?: { apiBase: string; env: 'dev' | 'prod' }
}
declare module 'old-print-lib' {
  export function print(html: string, opts?: { size?: string }): void
}
```

五、执行顺序（反了全是重复劳动）

- 先建 global.d.ts，把所有全局声明写清楚；
- tsconfig 允许 JS + TS 混跑（allowJs、checkJs: false）；
- 按模块分批，每批喂完标准答案再让 AI 补；
- 每批 tsc --noEmit 清零报错再进下一批。

完整版含每类报错占比与更多示例 · <https://github.com/zhongzht/fe-ai-cheatsheet>